République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

École Supérieure Privée

d'Ingénierie et de Technologies

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur

Spécialité : Génie Logiciel

Par

NAFFATI

Industrialisation DevSecOps d'une Application Web Cloud-Native

Encadrant professionnel:	**Sopra Steria**
Encadrant académique:	**ESPRIT**

Réalisé au sein de Sopra Steria

A.U : 2025 - 2026

Année Universitaire 2025 - 2026

Dédicace

À tous ceux qui croient que la sécurité n'est pas une fonctionnalité optionnelle, mais une responsabilité fondamentale de tout ingénieur logiciel.

À ma famille, pour leur soutien indéfectible tout au long de ce parcours.

Remerciements

Je tiens à remercier l'ensemble de l'équipe **Sopra Steria** pour l'accueil chaleureux, le soutien technique et la confiance accordée tout au long de ce projet de fin d'études.

Je remercie également mes encadrants académiques de **l'ESPRIT** pour leurs conseils précieux, leur rigueur scientifique et leur suivi attentif qui ont été déterminants dans la réalisation de ce travail.

Ma gratitude va aussi à tous mes collègues de promotion dont les échanges ont enrichi ma réflexion, ainsi qu'à ma famille pour son soutien moral constant.

Contents

List of Figures **viii**

List of Tables **ix**

Introduction Générale **1**

1 Contexte, Problématique et Étude de l'Existant **3**

1.1 Présentation de l'Organisme d'Accueil 4

1.1.1 Sopra Steria 4

1.1.2 Sopra HR Software 4

1.1.3 Département d'Accueil 5

1.2 Le Modèle «Deploy First, Secure Later» et ses Limites 5

1.3 Les Pressions Réglementaires et Industrielles 6

1.4 Problématique Centrale 6

1.5 Étude de l'Existant — L'Approche Jenkins 6

1.5.1 Description de l'Ancienne Architecture 6

1.5.2 Problèmes Identifiés 8

1.5.3 Dette Technique et Sécuritaire Accumulée 9

2 Analyse des Besoins **10**

2.1 Identification des Acteurs 11

2.2 Besoins Fonctionnels 11

2.3 Besoins Non Fonctionnels 13

2.4 Diagramme de Cas d'Utilisation Global 15

2.5 Cas d'Utilisation Détaillés 16

2.5.1 CU-01 : Pousser du Code et Déclencher le Pipeline 16

2.5.2 CU-02 : Déployer via GitOps 17

2.5.3 CU-03 : Gérer les Tâches Todo 18

2.6 Méthodologie de Travail : Scrum Agile 18

2.6.1 Choix de la Méthodologie 18

2.6.2 Équipe Scrum 19

2.6.3 Planification des Sprints 19
2.6.4 Product Backlog 20

3 Analyse et Conception du Système 23

3.1 Définition du DevSecOps et Principe Shift-Left 24
3.2 Les Huit Principes Directeurs du Projet 24
3.3 Architecture Globale en Six Couches 25
3.4 Architecture Physique GCP 26
3.5 Architecture Logique Kubernetes 27
3.6 Diagrammes de Conception 28
3.6.1 Diagramme de Séquence : Pipeline CI/CD Complet 28
3.6.2 Diagramme de Séquence : Authentification Workload Identity Federation 30
3.6.3 Diagramme d'Activité : Flux GitOps avec Argo CD 31
3.6.4 Diagramme de Déploiement 32
3.7 L'Application Cible : Todo App React 32
3.7.1 Description Fonctionnelle 32
3.7.2 Diagramme de Classes de l'Application 34
3.7.3 Stack Technique 34
3.7.4 Vulnérabilités Intentionnelles 35
3.8 Choix Technologiques et Justifications 37
3.9 Comparaison Globale Avant/Après 38

4 Implémentation et Réalisation par Sprints Agile 39

4.1 Sprint 1 : Infrastructure as Code et Cluster GKE 40
4.1.1 Objectif et Backlog du Sprint 40
4.1.2 Infrastructure as Code avec Terraform 40
4.1.3 Rétrospective Sprint 1 43
4.2 Sprint 2 : Containerisation et Pipeline CI/CD 44
4.2.1 Objectif et Backlog du Sprint 44
4.2.2 Containerisation et Sécurité des Images Docker 44
4.2.3 Pipeline CI/CD avec GitHub Actions 46
4.2.4 Orchestration Kubernetes sur GKE 48

4.2.5 Rétrospective Sprint 2 49
4.3 Sprint 3 : Sécurité Intégrée et GitOps 50
4.3.1 Objectif et Backlog du Sprint 50
4.3.2 SAST avec SonarCloud 50
4.3.3 SCA avec Trivy — Vulnérabilités Détectées 51
4.3.4 DAST avec OWASP ZAP 52
4.3.5 GitOps avec Argo CD 54
4.3.6 Réseau Zero Trust — NetworkPolicies 55
4.3.7 Rétrospective Sprint 3 56
4.4 Sprint 4 : Tests E2E, Observabilité, SIEM et Bilan 57
4.4.1 Objectif et Backlog du Sprint 57
4.4.2 Tests Unitaires avec Vitest 57
4.4.3 Tests E2E Playwright — 6 Scénarios 58
4.4.4 Observabilité avec Prometheus et Grafana 59
4.4.5 Sécurité Runtime — Wazuh SIEM 60
4.4.6 Gouvernance IAM et Identité 61
4.4.7 Rétrospective Sprint 4 61
4.5 Bilan Global : Résultats et Comparaison Avant/Après 63
4.5.1 Tableau de Bord de Sécurité 63
4.5.2 KPIs Techniques Mesurés 64
4.5.3 Alignement aux Frameworks de Sécurité 65

Conclusion et Perspectives **66**

Annexes **70**

A Commandes de Déploiement **70**
1.1 Provisionnement de l'Infrastructure 71
1.2 Deploiement des Ressources Kubernetes 71
1.3 Verification de l'Etat Argo CD 71
1.4 Lancement des Tests E2E 71
1.5 Consultation des Logs Wazuh 71

B Variables GitHub Actions et Ports des Services **72**

2.1 Secrets et Variables GitHub Actions 73

2.2 Ports et Services de la Plateforme 73

C Glossaire des Termes Techniques **74**

List of Figures

1.1 Vue globale de Sopra HR 5
1.2 Organigramme de Sopra HR Software 5
1.3 Architecture logique de l'approche Jenkins (état initial) 7

2.1 Diagramme de cas d'utilisation global de la plateforme DevSecOps 15
2.2 Processus Scrum appliqué au projet 19
2.3 Diagramme de Gantt de la planification des sprints 22

3.1 Principe Shift-Left de la sécurité dans le cycle DevSecOps 24
3.2 Architecture en six couches de la plateforme DevSecOps 26
3.3 Architecture physique de l'infrastructure GCP 27
3.4 Architecture logique du cluster Kubernetes 28
3.5 Diagramme de séquence du pipeline CI/CD complet 29
3.6 Diagramme de séquence de l'authentification Workload Identity Federation . . . 30
3.7 Diagramme d'activité du flux GitOps avec Argo CD 31
3.8 Diagramme de déploiement de la plateforme 32
3.9 Page de login de la Todo App 33
3.10 Dashboard de gestion des tâches Todo 33
3.11 Diagramme de classes de l'application Todo App 34

4.1 Sortie de `terraform plan` montrant les ressources à créer 43
4.2 Cluster GKE Autopilot provisionné via Terraform 43
4.3 Vue globale du pipeline GitHub Actions 47
4.4 Exécution réussie du job build-test 48
4.5 Dashboard SonarCloud montrant les résultats d'analyse 51
4.6 Résultats du scan Trivy montrant les CVEs de l'image nginx:1.21 52
4.7 Rapport OWASP ZAP montrant les findings avant durcissement nginx 53
4.8 Dashboard Argo CD montrant l'état synchronisé de l'application 54
4.9 Rapport de couverture des tests unitaires Vitest 58
4.10 Résultats des tests Playwright E2E (6/6 passed) 59

4.11 Dashboard Grafana montrant les métriques Nginx en temps réel 60
4.12 Dashboard Wazuh avec 4 agents opérationnels 60

List of Tables

1.1 Problèmes du modèle «Deploy First, Secure Later» 5
1.2 État du système avant la transformation DevSecOps 8
1.3 Couverture OWASP Top 10 avant ce projet 9

2.1 Acteurs du système et leurs rôles 11
2.2 Besoins fonctionnels du projet 12
2.3 Besoins non fonctionnels du projet 14
2.4 Description du cas d'utilisation CU-01 16
2.5 Description du cas d'utilisation CU-02 17
2.6 Description du cas d'utilisation CU-03 18
2.7 Composition de l'équipe Scrum 19
2.8 Planification des sprints du projet 20
2.9 Product Backlog (extrait) 21

3.1 Huit principes directeurs de la plateforme DevSecOps 25
3.2 Organisation en namespaces Kubernetes 27
3.3 Stack technique de l'application 35
3.4 Choix technologiques et justifications 37
3.5 Comparaison des approches avant et après transformation 38

4.1 Backlog du Sprint 1 40
4.2 Comparaison ClickOps vs Terraform IaC 41
4.3 Rétrospective Sprint 1 43
4.4 Backlog du Sprint 2 44
4.5 Vulnérabilités intentionnelles dans le Dockerfile 45
4.6 nginx:1.21 vs nginx:alpine 46
4.7 Rétrospective Sprint 2 49
4.8 Backlog du Sprint 3 50
4.9 Les trois niveaux de scan Trivy dans le pipeline 51
4.10 CVEs détectées par Trivy FS sur les dépendances npm 52

4.11 Findings OWASP ZAP sur l'application 53
4.12 Matrice des flux réseau autorisés 55
4.13 Rétrospective Sprint 3 56
4.14 Backlog du Sprint 4 57
4.15 Les 6 scénarios de tests Playwright E2E 58
4.16 Métriques Nginx collectées et leur utilité sécurité 59
4.17 Identités et rôles IAM du projet 61
4.18 Rétrospective Sprint 4 61
4.19 Bilan sécurité avant et après la transformation DevSecOps 63
4.20 Indicateurs clés de performance avant/après 64
4.21 Alignement aux frameworks de sécurité 65
4.22 Bilan quantitatif de la transformation DevSecOps 67
4.23 Compétences acquises durant le stage 68

B.1 Variables et secrets requis dans GitHub Actions 73
B.2 Ports et services de la plateforme DevSecOps 73

Introduction Générale

Contexte du Projet de Fin d'Études

Ce projet de fin d'études s'inscrit dans le cadre d'un stage d'ingénierie au sein de **Sopra Steria**, groupe international de conseil et de services numériques. La mission confiée est de moderniser et de sécuriser le cycle de vie logiciel d'une application web en adoptant une approche **DevSecOps** complète, depuis le code source jusqu'à la production sur le cloud.

La transformation numérique des entreprises impose aujourd'hui des exigences croissantes en matière de rapidité de livraison, de fiabilité des systèmes, et surtout de **sécurité intégrée**. Les organisations qui déploient des applications cloud-native font face à une complexité croissante : gestion de conteneurs, orchestration Kubernetes, multiplication des pipelines CI/CD, et une surface d'attaque qui s'étend à chaque couche de la chaîne de valeur.

Objectifs du Rapport

Ce rapport a pour but de :

- Documenter la **démarche complète** de conception et d'implémentation d'une plateforme DevSecOps ;
- Démontrer la **valeur ajoutée** par rapport à l'approche initiale basée sur Jenkins sans sécurité ;
- Présenter les **choix techniques** et les justifications architecturales ;
- Illustrer les **résultats concrets** à travers des métriques et artefacts ;
- Proposer un **plan d'amélioration continue** pour les phases suivantes.

Structure du Rapport

Le rapport est organisé de manière progressive selon quatre chapitres :

- Le **Chapitre 1** analyse le contexte, la problématique et l'état de l'existant (pipeline Jenkins). Il identifie les lacunes de sécurité et justifie la nécessité d'une transformation ;

- Le **Chapitre 2** présente l'analyse des besoins fonctionnels et non fonctionnels du projet, identifie les acteurs, définit les cas d'utilisation, et décrit la méthodologie Agile adoptée avec la planification des sprints ;
- Le **Chapitre 3** détaille l'analyse et la conception du système : l'architecture DevSecOps en six couches, l'architecture physique GCP, l'architecture logique Kubernetes, les diagrammes de séquence et d'activité, ainsi que les choix technologiques ;
- Le **Chapitre 4** décrit la réalisation du projet organisée en quatre sprints Agile : infrastructure et IaC (Sprint 1), containerisation et pipeline CI/CD (Sprint 2), sécurité intégrée et GitOps (Sprint 3), et tests automatisés, observabilité et résultats (Sprint 4).

Le rapport se conclut par une synthèse des compétences acquises, les limites identifiées et une roadmap d'amélioration continue à court, moyen et long terme.

Chapter 1

Contexte, Problématique et Étude de l'Existant

Sommaire

1 Présentation de l'Organisme d'Accueil 4

2 Le Modèle «Deploy First, Secure Later» et ses Limites 5

3 Les Pressions Réglementaires et Industrielles 6

4 Problématique Centrale 6

5 Étude de l'Existant — L'Approche Jenkins 6

Introduction

Ce chapitre pose les fondements du projet en présentant le contexte dans lequel il s'inscrit, la problématique identifiée en début de stage, ainsi qu'une analyse rigoureuse de l'existant. L'objectif est de justifier la nécessité d'une transformation profonde des pratiques de développement et de livraison logicielle, en s'appuyant sur des données concrètes et une comparaison avec les standards industriels actuels.

1.1 Présentation de l'Organisme d'Accueil

Sopra HR Software est une Société de Services et d'Ingénierie Informatique (SSII), filiale de Sopra Steria.

Dans cette section, nous présentons d'abord Sopra Steria, puis Sopra HR Software, et enfin le département d'accueil au sein duquel ce stage a été réalisé.

1.1.1 Sopra Steria

Sopra Steria est un acteur majeur de la transformation numérique sur le marché européen. Le groupe propose un portefeuille complet de services couvrant le conseil, l'intégration de systèmes, l'édition de solutions métiers, la gestion des infrastructures et les services de processus métier. Ce portefeuille répond aux enjeux de développement et de compétitivité des grandes entreprises.

La valeur ajoutée de Sopra Steria réside dans sa capacité à accompagner ses clients tout au long de leur transformation, en combinant innovation et performance opérationnelle.

Le groupe aide ses clients à tirer parti du numérique grâce à des logiciels métiers dans les domaines de la banque, des ressources humaines (notamment Pleiades et HR Access), ainsi que de la gestion immobilière (Ulis, Ikos et Altaix).

Le groupe Sopra Steria a enregistré un chiffre d'affaires de 3,8 milliards d'euros en 2017, avec plus de 42 000 collaborateurs dans plus de 20 pays [1].

1.1.2 Sopra HR Software

Sopra HR Software est une filiale de Sopra Steria spécialisée dans la gestion des ressources humaines, la paie et la gestion des talents aux niveaux national et international.

Sopra HR propose des solutions RH adaptées aux besoins des directions des ressources humaines des grandes et moyennes organisations, dans les secteurs public et privé.

Sopra HR est présent dans 10 pays, dont l'Allemagne, la Belgique, le Luxembourg, l'Italie, la Suisse, le Royaume-Uni, la Tunisie, le Maroc et l'Espagne, garantissant une proximité géographique et des services adaptés aux besoins des clients [2].

Espace réservé pour la Figure 1.1
Vue globale de Sopra HR

Figure 1.1: Vue globale de Sopra HR

1.1.3 Département d'Accueil

Afin d'assurer le bon déroulement des activités et de garantir un haut niveau de service, plusieurs départements collaborent au sein de Sopra HR.

Espace réservé pour la Figure 1.2
Organigramme de Sopra HR Software

Figure 1.2: Organigramme de Sopra HR Software

Notre stage de fin d'études a été réalisé au sein de l'équipe internationale R&D. Cette équipe a pour mission de rechercher, de développer et de proposer des solutions répondant à l'évolution des besoins des directeurs des ressources humaines, tout en anticipant les tendances et perspectives futures du domaine RH.

1.2 Le Modèle «Deploy First, Secure Later» et ses Limites

Pendant longtemps, les équipes de développement ont fonctionné selon un modèle où la sécurité était traitée comme une **étape finale**, voire optionnelle. On codait, on testait les fonctionnalités, on déployait et on pensait à la sécurité «plus tard», souvent lors d'un audit annuel ou après un incident.

Ce modèle présente des problèmes fondamentaux résumés dans le tableau 1.1 :

Tableau 1.1: Problèmes du modèle «Deploy First, Secure Later»

Problème	Conséquence
Vulnérabilités découvertes tard dans le cycle	Coût de correction 10x à 100x plus élevé
Secrets hardcodés dans le code	Risque de compromission de l'infrastructure
Images Docker non scannées	CVEs critiques en production
Accès réseau non restreints	Propagation latérale en cas d'intrusion
Pas de monitoring sécurité runtime	Détection des attaques impossible

1.3 Les Pressions Réglementaires et Industrielles

En 2026, les exigences réglementaires renforcent la nécessité d'une sécurité intégrée:

- **NIS2** (*Network and Information Security Directive*) impose des contrôles de sécurité sur les chaînes d'approvisionnement logicielles;
- **DORA** (*Digital Operational Resilience Act*) impose la résilience et la traçabilité des changements;
- **ISO 27001** et **SOC 2** sont devenus des critères de sélection des fournisseurs de services numériques;
- Les standards **SLSA** (*Supply-chain Levels for Software Artifacts*) définissent des niveaux d'intégrité des artefacts logiciels.

1.4 Problématique Centrale

La problématique de ce projet se formule ainsi:

> *Comment transformer un pipeline CI/CD basique (Jenkins + tests unitaires) en une chaîne DevSecOps complète qui intègre la sécurité à chaque étape, sans sacrifier la vélocité de livraison, et avec des preuves auditables de conformité ?*

Cette problématique est multidimensionnelle:

- **Technique:** Quels outils choisir? Comment les intégrer?
- **Organisationnelle:** Comment *shift-left* la sécurité dans les pratiques d'équipe?
- **Économique:** Comment démontrer le ROI de la sécurité intégrée?
- **Opérationnelle:** Comment maintenir la visibilité sur un système distribué complexe?

1.5 Étude de l'Existant — L'Approche Jenkins

1.5.1 Description de l'Ancienne Architecture

Avant ce projet, le système en place reposait sur une approche minimaliste et très courante dans les petites équipes: un serveur Jenkins auto-hébergé configuré et maintenu de manière manuelle.

Les jobs Jenkins étaient définis et administrés à la main par les développeurs et les builds étaient souvent lancés manuellement par un développeur plutôt que par des déclencheurs automatisés. Jenkins était relié à notre dépôt Git, mais l'intégration restait limitée: il se contentait d'exécuter des tests unitaires avant un déploiement manuel effectué via des scripts shell.

La figure 1.3 présente l'architecture logique de cette approche Jenkins avant la transformation DevSecOps.

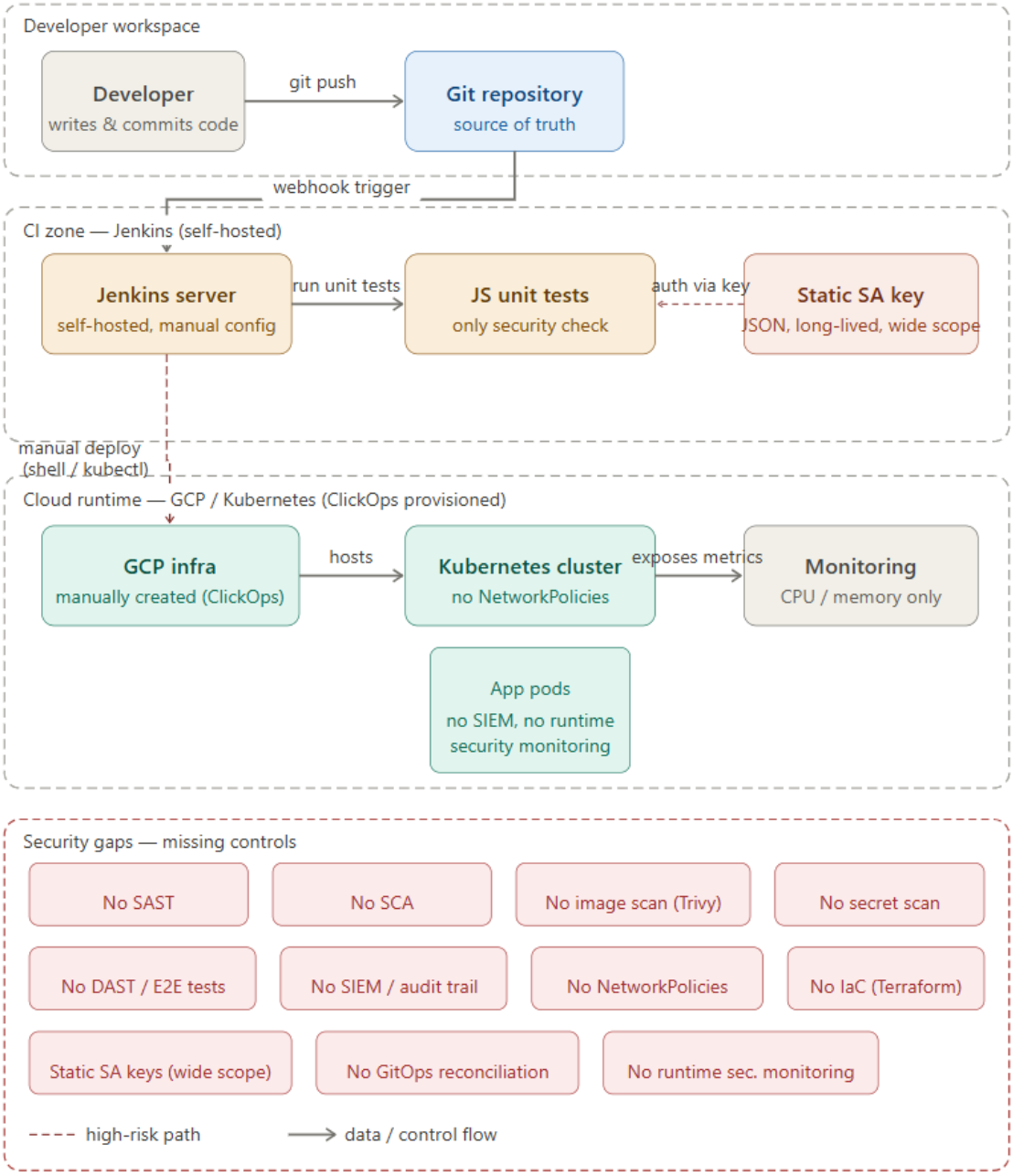

Figure 1.3: Architecture logique de l'approche Jenkins (état initial)

Le tableau 1.2 synthétise l'état de l'art avant la transformation:

Tableau 1.2: État du système avant la transformation DevSecOps

Aspect	État Avant
Outil CI	Jenkins (auto-hébergé, configuration manuelle)
Tests	Uniquement tests BDD
Sécurité	**Aucune**
Scan de vulnérabilités	Absent
Scan des images Docker	Absent
Scan des secrets	Absent
Tests E2E	Absents ou manuels
Infrastructure	Créée manuellement via console GCP (ClickOps)
Déploiement	Scripts shell ad-hoc, pas versionnés
Authentification CI	Clés JSON longue durée dans les secrets Jenkins
Monitoring	Basique (CPU/mémoire uniquement)
Sécurité réseau	Pas de NetworkPolicies
SIEM	Absent
Audit trail	Inexistant

1.5.2 Problèmes Identifiés

Problème 1 — Absence totale de sécurité dans le pipeline: Le pipeline Jenkins se contentait de vérifier que les tests passaient. Aucune analyse statique de sécurité (SAST), aucun scan de dépendances (SCA), aucun scan d'images Docker n'était effectué. Des vulnérabilités critiques pouvaient transiter du code source jusqu'en production sans être détectées.

Problème 2 — Clés d'authentification statiques: Jenkins utilisait des clés JSON Google Cloud Service Account avec des permissions larges, stockées en clair dans la configuration Jenkins. La compromission de ces clés aurait donné un accès complet à l'infrastructure GCP.

Problème 3 — Infrastructure non reproductible: Les ressources GCP (cluster Kubernetes, réseau, IAM) étaient créées manuellement via la console. Il était impossible de recréer l'environnement de façon fiable, et les modifications n'étaient pas tracées.

Problème 4 — Déploiements impératifs non versionnés: Les déploiements étaient effectués via des commandes `kubectl apply` ou des scripts shell non versionnés, sans historique, sans rollback automatique et sans validation de l'état désiré.

Problème 5 — Aucune visibilité sur la sécurité en production: Après le déploiement, aucun outil ne surveillait les comportements anormaux, les tentatives d'intrusion ou les événements de sécurité sur les nœuds Kubernetes.

Problème 6 — Tests non représentatifs: Les tests unitaires ne testaient que des fonctions isolées, sans jamais valider le comportement réel de l'application dans son contexte de déploiement.

1.5.3 Dette Technique et Sécuritaire Accumulée

Le tableau 1.3 résume la dette accumulée selon les catégories OWASP Top 10:

Tableau 1.3: Couverture OWASP Top 10 avant ce projet

Catégorie OWASP	État Avant
A01 - Broken Access Control	Non contrôlé (credentials hardcodés)
A02 - Cryptographic Failures	Secrets en clair dans le code et CI
A03 - Injection (XSS)	Non détecté, pas de SAST
A05 - Security Misconfiguration	Containers root, pas de NetworkPolicies
A06 - Vulnerable Components	Dépendances jamais auditées
A08 - Software Integrity Failures	Images non signées, pas de SBOM
A09 - Security Logging Failures	Aucun SIEM, aucun audit trail

Conclusion

Ce chapitre a établi le contexte général du projet en identifiant les six lacunes majeures de l'approche Jenkins initiale et en formulant la problématique centrale : intégrer la sécurité à chaque étape du cycle de livraison sans sacrifier la vélocité. L'analyse de la dette technique et sécuritaire accumulée, notamment au regard du référentiel OWASP Top 10, justifie pleinement l'ampleur de la transformation entreprise. Le chapitre suivant présente l'analyse des besoins fonctionnels et non fonctionnels identifiés pour répondre à cette problématique.

Chapter 2

Analyse des Besoins

Sommaire

1 Identification des Acteurs 11

2 Besoins Fonctionnels 11

3 Besoins Non Fonctionnels 13

4 Diagramme de Cas d'Utilisation Global 15

5 Cas d'Utilisation Détaillés 16

6 Méthodologie de Travail : Scrum Agile 18

Introduction

Ce chapitre présente l'analyse des besoins du projet. À partir de la problématique identifiée au chapitre précédent, nous identifions les acteurs impliqués, formulons les besoins fonctionnels et non fonctionnels, modélisons les cas d'utilisation, et décrivons la méthodologie Agile adoptée pour organiser la réalisation du projet en sprints itératifs.

2.1 Identification des Acteurs

Trois catégories d'acteurs interagissent avec la plateforme DevSecOps :

Tableau 2.1: Acteurs du système et leurs rôles

Acteur	Type	Rôle
Développeur	Humain	Écrit le code, pousse les commits, consulte les rapports de sécurité
Ingénieur DevOps / DevSecOps	Humain	Configure l'infrastructure, le pipeline et les politiques de sécurité
Pipeline CI/CD (GitHub Actions)	Système	Exécute automatiquement les builds, scans et déploiements
Argo CD	Système	Réconcilie l'état du cluster avec l'état désiré dans Git
Utilisateur final	Humain	Utilise l'application Todo App via le navigateur

2.2 Besoins Fonctionnels

Les besoins fonctionnels couvrent les capacités que la plateforme doit offrir. Ils sont organisés par domaine dans le tableau 2.2 :

Tableau 2.2: Besoins fonctionnels du projet

ID	Besoin Fonctionnel	Domaine
BF-01	Provisionner l'infrastructure GCP de manière reproductible via du code	Infrastructure
BF-02	Construire et publier automatiquement les images Docker à chaque push	CI/CD
BF-03	Exécuter des tests unitaires automatiquement à chaque commit	CI/CD
BF-04	Scanner le code source pour détecter les vulnérabilités (SAST)	Sécurité
BF-05	Scanner les dépendances npm et les images Docker pour les CVEs (SCA)	Sécurité
BF-06	Scanner l'application déployée en boîte noire (DAST)	Sécurité
BF-07	Déployer automatiquement l'application via GitOps (Argo CD)	Déploiement
BF-08	Exécuter des tests E2E dans le cluster après chaque déploiement	Tests
BF-09	Collecter et visualiser les métriques de l'application en temps réel	Observabilité
BF-10	Surveiller les événements de	Sécurité

2.3 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité de la plateforme :

Tableau 2.3: Besoins non fonctionnels du projet

ID	Besoin Non Fonctionnel	Catégorie
BNF-01	Zéro clé d'authentification statique dans le pipeline (Keyless)	Sécurité
BNF-02	Réseau Zero Trust : tout trafic interdit par défaut, flux explicites	Sécurité
BNF-03	Auto-scaling horizontal de 2 à 8 replicas selon la charge CPU	Performance
BNF-04	Infrastructure 100% reproductible à partir du code source	Fiabilité
BNF-05	Chaque build, scan et déploiement doit produire un rapport auditable	Auditabilité
BNF-06	Détection automatique et correction du drift de configuration (self-heal)	Résilience
BNF-07	Couverture de tests unitaires $\geq 70\%$	Qualité
BNF-08	Temps de détection d'une CVE critique < 5 minutes	Sécurité
BNF-09	Rollback automatique via Git revert + Argo CD	Disponibilité

2.4 Diagramme de Cas d'Utilisation Global

Le diagramme de cas d'utilisation global modélise les interactions entre les acteurs et les principales fonctionnalités de la plateforme DevSecOps.

Espace réservé pour la Figure 2.1

Diagramme de cas d'utilisation global

Acteurs : Développeur, Ingénieur DevSecOps, Utilisateur final,
Pipeline CI/CD (système), Argo CD (système)

Cas d'utilisation : Pousser du code, Construire & tester, Scanner la sécurité,
Déployer via GitOps, Surveiller, Gérer les tâches Todo

Figure 2.1: Diagramme de cas d'utilisation global de la plateforme DevSecOps

2.5 Cas d'Utilisation Détaillés

2.5.1 CU-01 : Pousser du Code et Déclencher le Pipeline

Tableau 2.4: Description du cas d'utilisation CU-01

Champ	**Description**
Titre	Pousser du code et déclencher le pipeline
Acteur principal	Développeur
Précondition	Le développeur a un accès au dépôt Git
Scénario nominal	1. Le développeur pousse un commit sur la branche principale 2. GitHub Actions détecte le push et démarre le pipeline 3. Le job `build-test` exécute lint, tests unitaires, SonarCloud et Trivy FS 4. En cas de succès, le job `app-image-release` construit et publie l'image Docker 5. Le job `update-k8s-tags` met à jour les manifests Kubernetes
Postcondition	L'image Docker est publiée, les manifests K8s sont mis à jour dans Git
Scénario alternatif	Si un test échoue ou un scan critique bloque, le pipeline s'arrête et notifie le développeur

2.5.2 CU-02 : Déployer via GitOps

Tableau 2.5: Description du cas d'utilisation CU-02

Champ	**Description**
Titre	Déployer automatiquement via GitOps
Acteur principal	Argo CD (système)
Précondition	Les manifests K8s dans Git ont été mis à jour avec le nouveau tag d'image
Scénario nominal	1. Argo CD détecte un changement dans le dépôt Git 2. Argo CD compare l'état désiré (Git) avec l'état réel du cluster 3. Argo CD synchronise automatiquement les ressources Kubernetes 4. Les hooks PostSync lancent les tests Playwright E2E 5. Le dashboard Argo CD affiche l'état synchronisé
Postcondition	L'application est déployée, les tests E2E sont validés
Scénario alternatif	En cas de drift détecté, Argo CD applique le self-heal automatiquement

2.5.3 CU-03 : Gérer les Tâches Todo

Tableau 2.6: Description du cas d'utilisation CU-03

Champ	Description
Titre	Gérer les tâches Todo (CRUD)
Acteur principal	Utilisateur final
Précondition	L'utilisateur est authentifié sur l'application
Scénario nominal	1. L'utilisateur ajoute une nouvelle tâche 2. L'utilisateur coche une tâche comme complétée 3. L'utilisateur supprime une tâche 4. Les données persistent via localStorage après rechargement
Postcondition	Les tâches sont sauvegardées et visibles
Scénario alternatif	Si l'utilisateur n'est pas authentifié, il est redirigé vers la page de login

2.6 Méthodologie de Travail : Scrum Agile

2.6.1 Choix de la Méthodologie

Le projet suit la méthodologie **Scrum** pour sa capacité à gérer la complexité de manière itérative et incrémentale. Chaque sprint livre un incrément fonctionnel et testable, permettant de valider progressivement les composantes de la plateforme DevSecOps.

Espace réservé pour la Figure 2.2

Processus Scrum appliqué au projet

Product Backlog → Sprint Planning → Sprint (2–3 semaines)
→ Sprint Review → Sprint Retrospective → Incrément livré

Figure 2.2: Processus Scrum appliqué au projet

2.6.2 Équipe Scrum

Tableau 2.7: Composition de l'équipe Scrum

Rôle Scrum	Personne	Responsabilité
Product Owner	Encadrant professionnel (Sopra Steria)	Définition des priorités et validation des livrables
Scrum Master	Encadrant académique (ESPRIT)	Suivi méthodologique et levée des blocages
Développeur	NAFFATI	Conception, implémentation et tests

2.6.3 Planification des Sprints

Le projet est découpé en **quatre sprints** de 2 à 3 semaines chacun, couvrant l'ensemble des besoins identifiés :

Tableau 2.8: Planification des sprints du projet

Sprint	Durée	Objectif	Besoins couverts
Sprint 1	3 semaines	Infrastructure as Code et cluster GKE	BF-01, BNF-04
Sprint 2	3 semaines	Containerisation, pipeline CI/CD et authentification Keyless	BF-02, BF-03, BNF-01, BNF-05
Sprint 3	3 semaines	Sécurité intégrée (SAST, SCA, DAST), GitOps et réseau Zero Trust	BF-04, BF-05, BF-06, BF-07, BNF-02, BNF-06
Sprint 4	2 semaines	Tests E2E, observabilité, SIEM et bilan	BF-08, BF-09, BF-10, BNF-03, BNF-07, BNF-08, BNF-09

2.6.4 Product Backlog

Le Product Backlog regroupe l'ensemble des *User Stories* priorisées par le Product Owner. Le tableau 2.9 présente les éléments principaux :

Tableau 2.9: Product Backlog (extrait)

ID	User Story	Priorité	Sprint
US-01	En tant que DevOps, je veux provisionner l'infra via Terraform	Haute	1
US-02	En tant que DevOps, je veux un cluster GKE Autopilot opérationnel	Haute	1
US-03	En tant que Dev, je veux un Dockerfile multi-stage optimisé	Haute	2
US-04	En tant que Dev, je veux un pipeline CI qui build et teste à chaque push	Haute	2
US-05	En tant que DevSecOps, je veux une auth CI sans clé statique (WIF)	Haute	2
US-06	En tant que DevSecOps, je veux des scans SAST/SCA automatiques	Haute	3
US-07	En tant que DevSecOps, je veux un déploiement GitOps avec Argo CD	Haute	3
US-08	En tant que DevSecOps, je veux	Moyenne	3

Espace réservé pour la Figure 2.3

Diagramme de Gantt du projet

Sprint 1 (semaines 1–3) | Sprint 2 (semaines 4–6) |

Sprint 3 (semaines 7–9) | Sprint 4 (semaines 10–11)

Figure 2.3: Diagramme de Gantt de la planification des sprints

Conclusion

Ce chapitre a formalisé les besoins du projet à travers onze besoins fonctionnels et neuf besoins non fonctionnels, identifié les acteurs du système, et modélisé les cas d'utilisation clés. La méthodologie Scrum adoptée organise la réalisation en quatre sprints itératifs, chacun livrant un incrément fonctionnel et testable. Le chapitre suivant présente l'analyse et la conception détaillée du système.

Chapter 3

Analyse et Conception du Système

Sommaire

1 Définition du DevSecOps et Principe Shift-Left 24

2 Les Huit Principes Directeurs du Projet 24

3 Architecture Globale en Six Couches 25

4 Architecture Physique GCP 26

5 Architecture Logique Kubernetes 27

6 Diagrammes de Conception 28

7 L'Application Cible : Todo App React 32

8 Choix Technologiques et Justifications 37

9 Comparaison Globale Avant/Après 38

Introduction

Ce chapitre présente l'analyse et la conception détaillée du système. Nous y définissons les fondements du DevSecOps et le principe Shift-Left, décrivons l'architecture globale en six couches, l'architecture physique sur Google Cloud Platform, l'architecture logique Kubernetes, les diagrammes UML de conception (séquence, activité, déploiement), le stack technique de l'application cible, et les choix technologiques justifiés.

3.1 Définition du DevSecOps et Principe Shift-Left

DevSecOps est une extension de la philosophie DevOps qui intègre la sécurité (**Sec**) comme responsabilité partagée, continue et automatisée dans le cycle de développement. L'idée fondamentale est le **Shift-Left** : déplacer les contrôles de sécurité le plus tôt possible dans le cycle, là où les corrections sont les moins coûteuses.

Espace réservé pour la Figure 3.1

Principe Shift-Left de la sécurité dans le cycle DevSecOps

Plan → Code → Build → Test → Release → Deploy → Monitor

Sec Review | SAST/Lint | SCA/Trivy | DAST/ZAP | cosign | K8s Sec | SIEM/Wazuh

Figure 3.1: Principe Shift-Left de la sécurité dans le cycle DevSecOps

3.2 Les Huit Principes Directeurs du Projet

Le tableau 3.1 présente les huit principes directeurs qui guident la conception de la solution :

Tableau 3.1: Huit principes directeurs de la plateforme DevSecOps

#	Principe	Implémentation
1	Tout est Code	Terraform pour l'infra, YAML pour K8s, tout versionné dans Git
2	Sécurité Shift-Left	Trivy + SonarCloud dès le build, avant tout déploiement
3	Keyless First	Workload Identity Federation — zéro clé JSON statique
4	Zero Trust Réseau	NetworkPolicies default-deny, flux explicitement autorisés
5	GitOps	Argo CD comme seul moteur de réconciliation du cluster
6	Tests au plus proche du réel	Playwright E2E dans le cluster, PostSync hooks
7	Observabilité Continue	Prometheus + Grafana + Wazuh en permanence
8	Preuves Auditables	Chaque outil génère des rapports uploadés comme artefacts CI

3.3 Architecture Globale en Six Couches

La solution s'articule autour de **six couches complémentaires** illustrées dans la figure 3.2 :

Espace réservé pour la Figure 3.2

Architecture en six couches de la plateforme DevSecOps

Couche 6 : Sécurité Runtime (Wazuh, NetworkPolicies, Workload Identity)

Couche 5 : Observabilité (Prometheus, Grafana, nginx-exporter)

Couche 4 : Orchestration (GKE Autopilot, Namespaces, HPA, Ingress)

Couche 3 : Livraison (GitHub Actions, Argo CD, Artifact Registry)

Couche 2 : Sécurité Applicative (Trivy, OWASP ZAP, SonarCloud, Playwright)

Couche 1 : Infrastructure (Terraform, GCP VPC, GKE, IAM, Artifact Registry)

Figure 3.2: Architecture en six couches de la plateforme DevSecOps

3.4 Architecture Physique GCP

L'infrastructure est déployée sur le projet GCP `pfe-esprit-489411` dans la région `europe-west1`. Elle comprend les composants suivants :

- **VPC `devops-vpc`** : réseau dédié avec subnet `devops-subnet` ; plages secondaires pour les nœuds (`10.10.0.0/20`), pods (`10.20.0.0/16`) et services (`10.30.0.0/20`) ;
- **Cluster GKE Autopilot `devops-cluster`** : mode géré, patches de sécurité automatiques, facturation à la ressource pod ;
- **Artifact Registry** : stockage des images Docker de l'application et des tests ;
- **VM Compute Engine** : hébergement du manager Wazuh SIEM (`e2-medium`, IP `10.10.0.2`).

Espace réservé pour la Figure 3.3

Architecture physique GCP

GCP Project → VPC devops-vpc → Subnet devops-subnet

GKE Autopilot Cluster | Artifact Registry | Wazuh VM (e2-medium)

Workload Identity Federation ↔ GitHub Actions

Figure 3.3: Architecture physique de l'infrastructure GCP

3.5 Architecture Logique Kubernetes

Le cluster est segmenté en **cinq namespaces fonctionnels** avec des responsabilités strictement séparées, comme présenté dans le tableau 3.2 :

Tableau 3.2: Organisation en namespaces Kubernetes

Namespace	Rôle	Ressources principales
`app`	Workload applicatif	Deployment, Service, Ingress, HPA, PostSync Job
`testing`	Validation E2E	Playwright Job, ServiceAccount
`observability`	Collecte de métriques	Prometheus, Grafana, ServiceMonitor
`security`	Politiques et SIEM	NetworkPolicies, Wazuh Agent DaemonSet
`argocd`	Moteur GitOps	Argo CD Applications

Espace réservé pour la Figure 3.4

Diagramme d'architecture logique Kubernetes

5 namespaces : app, testing, observability, security, argocd

Flux réseau : testing → app (TCP 80), observability → app (TCP 9113),

security → Wazuh VM (TCP 1514), Ingress → app (TCP 80)

Figure 3.4: Architecture logique du cluster Kubernetes

3.6 Diagrammes de Conception

3.6.1 Diagramme de Séquence : Pipeline CI/CD Complet

Le diagramme de séquence suivant modélise le flux complet du pipeline CI/CD, depuis le push du développeur jusqu'au déploiement GitOps et les tests E2E.

Espace réservé pour la Figure 3.5

Diagramme de séquence : Pipeline CI/CD complet

Développeur → GitHub : push commit

GitHub → Actions : déclencher pipeline

Actions → SonarCloud : scan SAST

Actions → Trivy : scan FS (SCA)

Actions → Artifact Registry : push image Docker

Actions → Trivy : scan image

Actions → Git : update K8s tags

Argo CD → Git : detect change

Argo CD → GKE : sync ressources

Argo CD → Playwright : PostSync E2E tests

Actions → ZAP : scan DAST

Figure 3.5: Diagramme de séquence du pipeline CI/CD complet

3.6.2 Diagramme de Séquence : Authentification Workload Identity Federation

Espace réservé pour la Figure 3.6

Diagramme de séquence : Authentification WIF

GitHub Actions → GitHub OIDC : request JWT token

GitHub Actions → GCP WIF Pool : present JWT

GCP WIF → GCP IAM : verify issuer, repo, branch

GCP IAM → GitHub Actions : temporary credentials (1h)

GitHub Actions → GCP APIs : authenticated calls

Figure 3.6: Diagramme de séquence de l'authentification Workload Identity Federation

3.6.3 Diagramme d'Activité : Flux GitOps avec Argo CD

Espace réservé pour la Figure 3.7

Diagramme d'activité : Flux GitOps Argo CD

[Commit dans Git] → Argo CD détecte le changement →

Comparer état désiré vs état réel → [Si différence] Synchroniser →

Appliquer les manifests K8s → Exécuter PostSync hooks (Playwright) →

Si drift détecté

Self-heal automatique → État synchronisé

Figure 3.7: Diagramme d'activité du flux GitOps avec Argo CD

3.6.4 Diagramme de Déploiement

Espace réservé pour la Figure 3.8

Diagramme de déploiement UML

Nœud GitHub : Actions Runner, OIDC Provider

Nœud GCP : VPC devops-vpc

GKE Cluster : ns app (Deployment + nginx-exporter sidecar),

ns testing (Playwright Job), ns observability (Prometheus, Grafana),

ns security (Wazuh Agent), ns argocd (Argo CD)

VM Wazuh : Wazuh Manager (TCP 1514)

Artifact Registry : Docker images

Nœud Externe : Navigateur utilisateur → Ingress GCE

Figure 3.8: Diagramme de déploiement de la plateforme

3.7 L'Application Cible : Todo App React

3.7.1 Description Fonctionnelle

L'application déployée est une **Todo App** avec un portail de connexion, développée avec React 18 et TypeScript. Bien que l'application soit fonctionnellement simple, elle sert de support idéal pour illustrer tous les aspects DevSecOps car elle possède :

- Une **authentification** (cible pour les tests de sécurité) ;
- Une **interaction utilisateur** (cible pour les tests E2E et XSS) ;
- Une **persistance de données** via localStorage (cible pour les vulnérabilités de stockage) ;

- Une **chaîne de dépendances npm** (cible pour le SCA).

Espace réservé pour la Figure 3.9

Capture d'écran : Page de login de la Todo App

Figure 3.9: Page de login de la Todo App

Espace réservé pour la Figure 3.10

Capture d'écran : Dashboard Todo avec liste de tâches

Figure 3.10: Dashboard de gestion des tâches Todo

3.7.2 Diagramme de Classes de l'Application

Espace réservé pour la Figure 3.11

Diagramme de classes de la Todo App

Classes : App, AuthContext, AuthProvider, LoginPage,

TodoDashboard, TodoItem, Todo (interface)

Relations : App → AuthProvider → LoginPage / TodoDashboard

TodoDashboard → TodoItem[] , TodoItem → Todo

Figure 3.11: Diagramme de classes de l'application Todo App

3.7.3 Stack Technique

Le tableau 3.3 détaille les technologies utilisées :

Tableau 3.3: Stack technique de l'application

Couche	Technologie	Version
Framework UI	React	18.3.1
Langage	TypeScript	5.8.3
Build tool	Vite	8.0.0
Styling	Tailwind CSS + Rosé Pine theme	3.4.17
Composants UI	shadcn/ui (Radix UI)	—
Animations	Framer Motion	12.x
Routing	React Router DOM	6.30.1
Gestionnaire de paquets	Bun	Latest
Serveur de production	Nginx	1.21 (vulnérable intentionnel)

3.7.4 Vulnérabilités Intentionnelles

Le composant `TodoItem` utilise intentionnellement `dangerouslySetInnerHTML` pour rendre le texte des tâches, créant une vulnérabilité **XSS stockée** détectable par OWASP ZAP et SonarCloud. De même, l'authentification est hardcodée côté client dans `AuthContext.tsx`. Ces choix sont délibérés pour la démonstration des outils DevSecOps.

3.8 Choix Technologiques et Justifications

Tableau 3.4: Choix technologiques et justifications

Domaine	Outil choisi	Justification
IaC	Terraform	Standard industriel, multi-cloud, état déclaratif
Orchestration	GKE Autopilot	Gestion nœuds par Google, sécurité par défaut, scaling auto
CI/CD	GitHub Actions	Natif GitHub, OIDC intégré, marketplace riche
GitOps	Argo CD	Self-heal, drift detection, UI dashboard, CNCF graduated
SAST	SonarCloud	Analyse TypeScript/React, Quality Gates, gratuit open-source
SCA + Container	Trivy	3 modes (fs, image, config), SARIF output, open-source
DAST	OWASP ZAP	Standard industriel, baseline scan intégrable en CI
SIEM	Wazuh	Open-source, HIDS + FIM + compliance, compatible K8s
Observabilité	Prometheus + Grafana	Standard CNCF, écosystème exporters, dashboards
Tests E2E	Playwright	Multi-navigateur, conteneurisable, tags pour parallélisme

3.9 Comparaison Globale Avant/Après

Le tableau 3.5 offre une vue synthétique de la transformation conçue :

Tableau 3.5: Comparaison des approches avant et après transformation

Dimension	Avant (Jenkins)	Après (DevSecOps)
Sécurité	Post-déploiement ou absente	Intégrée à chaque étape
Infrastructure	ClickOps manuel	Terraform IaC versionné
Déploiement	Scripts impératifs	GitOps déclaratif (Argo CD)
Tests	Unitaires seulement	Unit + E2E in-cluster + scans sécurité
Authentification	Clés JSON statiques	OIDC Workload Identity Federation
Réseau	Ouvert par défaut	Zero Trust (default-deny)
Monitoring	CPU/RAM basique	Métriques + SIEM + alertes
Audit trail	Inexistant	Chaque commit, build, déploiement tracé
Rollback	Manuel	Automatique via Git revert + Argo CD

Conclusion

Ce chapitre a présenté la conception détaillée du système : une architecture DevSecOps en six couches articulée autour de huit principes directeurs, une infrastructure GCP bien définie, un cluster Kubernetes segmenté en cinq namespaces, et des diagrammes UML modélisant les flux de livraison, de sécurité et de déploiement. Le chapitre suivant se consacre à l'implémentation concrète de cette conception, organisée en quatre sprints Agile.

Chapter 4

Implémentation et Réalisation par Sprints Agile

Sommaire

1 Sprint 1 : Infrastructure as Code et Cluster GKE 40

2 Sprint 2 : Containerisation et Pipeline CI/CD 44

3 Sprint 3 : Sécurité Intégrée et GitOps 50

4 Sprint 4 : Tests E2E, Observabilité, SIEM et Bilan 57

5 Bilan Global : Résultats et Comparaison Avant/Après 63

Introduction

Ce chapitre décrit la réalisation concrète du projet, organisée en quatre sprints Agile. Chaque sprint est présenté avec son backlog, les tâches réalisées, les extraits de code ou de configuration pertinents, les captures d'écran des résultats, et une rétrospective. Le chapitre se conclut par le bilan quantitatif global et les perspectives d'amélioration.

4.1 Sprint 1 : Infrastructure as Code et Cluster GKE

4.1.1 Objectif et Backlog du Sprint

Tableau 4.1: Backlog du Sprint 1

User Story	Tâche	Statut
US-01 : Provisionner l'infra via Terraform	Écrire les modules Terraform (main, network, gke, iam, wif, wazuh)	Terminé
US-01	Configurer le backend GCS pour le state distant	Terminé
US-02 : Cluster GKE Autopilot	Provisionner le cluster avec Workload Identity activé	Terminé
US-02	Provisionner la VM Wazuh Manager	Terminé
US-02	Créer les namespaces Kubernetes	Terminé

4.1.2 Infrastructure as Code avec Terraform

4.1.2.1 Justification du Choix Terraform

Face à l'ancienne approche ClickOps, **Terraform** a été choisi pour provisionner l'ensemble des ressources GCP de façon reproductible, versionnée et auditable. Le tableau 4.2 compare les deux approches :

Tableau 4.2: Comparaison ClickOps vs Terraform IaC

Critère	ClickOps	Terraform IaC
Reproductibilité	Non	Oui — `terraform apply` recrée tout
Versioning	Non	Git — chaque changement est tracé
Rollback	Non	`terraform destroy` + re-apply
Documentation	Non	Le code *est* la documentation
Audit trail	Non	`terraform plan` + historique Git
Collaboration	Difficile	PR reviews sur les changements infra

4.1.2.2 Structure des Modules Terraform

Le répertoire `terraform/` est organisé en modules spécialisés :

- `main.tf` : provider GCP, backend GCS (state distant), versions ;
- `variables.tf` : variables paramétrables (`project_id`, `region`, `repo_name`) ;
- `outputs.tf` : sorties utiles pour le pipeline (endpoint cluster, URL Artifact Registry) ;
- `apis.tf` : activation des APIs GCP nécessaires ;
- `network.tf` : VPC `devops-vpc`, subnet, plages secondaires ;
- `gke.tf` : cluster GKE Autopilot `devops-cluster` ;
- `iam.tf` : comptes de service, bindings IAM minimaux ;
- `wif.tf` : Workload Identity Federation (GitHub → GCP) ;
- `wazuh.tf` : VM `e2-medium` Wazuh Manager + règles firewall.

4.1.2.3 Ressources Clés Provisionnées

Cluster GKE Autopilot (`gke.tf`) :

```
resource "google_container_cluster" "main" {
  name     = "devops-cluster"
  location = var.region    # europe-west1
  enable_autopilot = true # Mode manage - pas de gestion de noeuds
  workload_identity_config {
    workload_pool = "${var.project_id}.svc.id.goog"
  }
  release_channel {
    channel = "REGULAR" # MAJ Kubernetes automatiques
  }
}
```

Listing 4.1: Configuration Terraform du cluster GKE Autopilot

Les avantages de GKE Autopilot sont : scaling automatique des nœuds selon la charge, patches de sécurité gérés par Google, profil de sécurité renforcé par défaut (restricted Pod Security Standards), et facturation à la ressource pod.

VM Wazuh Manager (`wazuh.tf`) : instance `e2-medium` Ubuntu 22.04, 50 Go de disque, bootstrappée automatiquement via un script de démarrage.

4.1.2.4 Bonnes Pratiques Terraform Appliquées

- **Backend distant :** state stocké dans GCS avec versioning activé ;
- **Dépendances explicites :** `depends_on` pour les ressources nécessitant une API activée ;
- **Variables externalisées :** `project_id`, `region` paramétrables ;
- **Moindre privilège IAM :** chaque compte de service reçoit uniquement les rôles strictement nécessaires.

Espace réservé pour la Figure 4.1

Capture d'écran : Sortie de `terraform plan`

Figure 4.1: Sortie de `terraform plan` montrant les ressources à créer

Espace réservé pour la Figure 4.2

Capture d'écran : Cluster GKE Autopilot dans la console GCP

Figure 4.2: Cluster GKE Autopilot provisionné via Terraform

4.1.3 Rétrospective Sprint 1

Tableau 4.3: Rétrospective Sprint 1

Catégorie	Détail
Ce qui a bien marché	Infrastructure 100% reproductible, Terraform plan clair
Difficultés	Configuration initiale du WIF provider, activation séquentielle des APIs GCP
Livrable	Cluster GKE + VPC + IAM + VM Wazuh opérationnels

4.2 Sprint 2 : Containerisation et Pipeline CI/CD

4.2.1 Objectif et Backlog du Sprint

Tableau 4.4: Backlog du Sprint 2

User Story	Tâche	Statut
US-03 : Dockerfile multi-stage	Écrire le Dockerfile avec build Bun + runner Nginx	Terminé
US-03	Introduire les vulnérabilités intentionnelles (nginx:1.21, secrets ENV)	Terminé
US-04 : Pipeline CI	Configurer GitHub Actions (build-test, app-image-release)	Terminé
US-05 : Auth Keyless (WIF)	Configurer l'authentification OIDC dans le pipeline	Terminé
US-04	Configurer le job update-k8s-tags	Terminé

4.2.2 Containerisation et Sécurité des Images Docker

4.2.2.1 Build Multi-Étapes

Le `Dockerfile` utilise un **build multi-étapes** pour minimiser la taille de l'image finale et réduire la surface d'attaque :

```
# Stage 1 : Build avec Bun
FROM oven/bun:1 AS builder
WORKDIR /app
COPY package.json bun.lock ./
RUN bun install
COPY . .
RUN bun run build
```

```
8  # --> dist/ contient les assets statiques compiles
9  # Stage 2 : Serveur Nginx minimal
10 FROM nginx:1.21 AS runner   # Attention : version vulnerable (demo)
11 COPY --from=builder /app/dist /usr/share/nginx/html
12 COPY ./nginx.conf /etc/nginx/conf.d/default.conf
13 EXPOSE 80
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 4.2: Dockerfile multi-étapes (version vulnérable pour démo)

4.2.2.2 Vulnérabilités Intentionnelles dans le Dockerfile

Tableau 4.5: Vulnérabilités intentionnelles dans le Dockerfile

Vulnérabilité	Ligne	Outil	Sévérité
Image nginx obsolète (CVEs OS)	`FROM nginx:1.21`	Trivy	CRITICAL/HIGH
Secrets hardcodés en ENV	`ENV APP_SECRET=...`	Trivy	HIGH
Fichier .env copié dans l'image	`COPY .env ...`	Trivy	HIGH
Exécution en root (pas de USER)	Absent	Trivy DS002	MEDIUM
Pas de HEALTHCHECK	Absent	Trivy DS026	LOW

4.2.2.3 Comparaison Image Vulnérable vs Durcie

Tableau 4.6: nginx:1.21 vs nginx:alpine

Aspect	nginx:1.21 (vulnérable)	nginx:alpine (durci)
CVEs OS critiques	Plusieurs dizaines	Quasi-inexistants
Taille image	∼133 MB	∼23 MB
Utilisateur	root	non-root possible
Surface d'attaque	Large	Minimale

4.2.3 Pipeline CI/CD avec GitHub Actions

4.2.3.1 Vue Globale du Pipeline

Le pipeline GitHub Actions est déclenché à chaque *push* sur la branche principale. Il s'articule en plusieurs jobs interdépendants :

1. **build-test** : tests unitaires, lint, SonarCloud, Trivy FS ;
2. **prepare-release-infra** : parallèle app-image-release et playwright-release ;
3. **app-image-release** : build image Docker, push vers Artifact Registry, scan Trivy ;
4. **update-k8s-tags** : commit du nouveau SHA dans les manifests Kubernetes ;
5. **e2e-tests** : matrice de 6 jobs Playwright (@t1 à @t6) ;
6. **zap-baseline** : scan DAST OWASP ZAP post-déploiement.

Espace réservé pour la Figure 4.3

Capture d'écran : Vue globale du pipeline GitHub Actions

Montrant les jobs : build-test → app-image-release → update-k8s-tags → e2e-tests → zap-baseline

Figure 4.3: Vue globale du pipeline GitHub Actions

4.2.3.2 Job build-test — Première Ligne de Défense

Le job `build-test` constitue la premiere ligne de defense securitaire :

```yaml
build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: oven-sh/setup-bun@v2
    - run: bun install
    - run: bun run lint
    - run: bun run test:coverage
    - name: SonarCloud Scan
      uses: SonarSource/sonarcloud-github-action@master
      env:
        SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
    - name: Trivy FS Scan
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'fs'
        format: 'sarif'
        output: 'trivy-fs-results.sarif'
        scanners: 'vuln,secret,misconfig'
    - uses: github/codeql-action/upload-sarif@v3
```

```
21        with:
22          sarif_file: 'trivy-fs-results.sarif'
```

Listing 4.3: Extrait du job build-test GitHub Actions

4.2.3.3 Authentification sans Clés Statiques — Workload Identity Federation

L'un des apports majeurs de ce projet est le remplacement des clés JSON statiques par l'authentification **OIDC via Workload Identity Federation** :

```
1 # Avant : secrets.GCP_SA_KEY (cle JSON longue duree)
2 # Apres :
3 - name: Authenticate to GCP via WIF
4   uses: google-github-actions/auth@v2
5   with:
6     workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
7     service_account: ${{ secrets.WIF_SERVICE_ACCOUNT }}
```

Listing 4.4: Authentification GCP sans clé statique

GitHub Actions obtient un **token OIDC éphémère** valide 1 heure maximum, échangé contre des credentials GCP temporaires. Aucune clé persistante n'est stockée.

Espace réservé pour la Figure 4.4

Capture d'écran : Job build-test réussi dans GitHub Actions

Figure 4.4: Exécution réussie du job build-test

4.2.4 Orchestration Kubernetes sur GKE

Le déploiement définit **deux containers dans le même pod** :

- **Container `app`** : image Nginx servant l'application React compilée, port 80, ressources `cpu: 100m`, `memory: 128Mi` ;

- **Container `nginx-exporter`** (sidecar) : image `nginx/nginx-prometheus-exporter:1.1.0`, expose les métriques Nginx en format Prometheus sur le port 9113.

L'HPA maintient **2 replicas minimum** et scale jusqu'à **8 replicas** à 60% d'utilisation CPU. Un Ingress GCE expose l'application sur Internet.

4.2.5 Rétrospective Sprint 2

Tableau 4.7: Rétrospective Sprint 2

Catégorie	Détail
Ce qui a bien marché	Pipeline CI fonctionnel dès le premier push, WIF opérationnel
Difficultés	Configuration OIDC WIF (concepts peu documentés), orchestration des jobs GitHub Actions
Livrable	Pipeline CI/CD complet, image Docker publiée, app déployée sur GKE

4.3 Sprint 3 : Sécurité Intégrée et GitOps

4.3.1 Objectif et Backlog du Sprint

Tableau 4.8: Backlog du Sprint 3

User Story	Tâche	Statut
US-06 : Scans SAST/SCA	Intégrer SonarCloud et Trivy (3 niveaux) dans le pipeline	Terminé
US-07 : GitOps Argo CD	Configurer l'Application Argo CD avec auto-sync et self-heal	Terminé
US-07	Configurer les PostSync hooks pour Playwright	Terminé
US-08 : Zero Trust	Créer les 12 NetworkPolicies default-deny	Terminé
US-09 : DAST ZAP	Intégrer le scan OWASP ZAP baseline post-déploiement	Terminé

4.3.2 SAST avec SonarCloud

SonarCloud analyse le code TypeScript/React à chaque push et détecte :

- **Security Hotspots** : utilisations de `dangerouslySetInnerHTML` (XSS) ;
- **Vulnerabilities** : patterns de code pouvant mener à des injections ;
- **Code Smells** : indicateurs de qualité masquant des vulnérabilités ;
- **Coverage gate** : bloque la PR si la couverture de tests chute sous le seuil.

Espace réservé pour la Figure 4.5

Capture d'écran : Dashboard SonarCloud du projet

Montrant les security hotspots, la couverture, les code smells

Figure 4.5: Dashboard SonarCloud montrant les résultats d'analyse

4.3.3 SCA avec Trivy — Vulnérabilités Détectées

Le tableau 4.9 détaille les trois niveaux de scan Trivy intégrés :

Tableau 4.9: Les trois niveaux de scan Trivy dans le pipeline

Scan	Cible	Ce qui est détecté
`trivy fs`	Code source + `package.json`	Vulns deps npm, secrets dans fichiers
`trivy image`	Image Docker construite	CVEs OS, secrets ENV
`trivy config`	Manifests YAML Kubernetes	Misconfigurations IaC

Tableau 4.10: CVEs détectées par Trivy FS sur les dépendances npm

CVE	Package	Sévérité	Description
CVE-2021-23337	lodash 4.17.20	HIGH	Command Injection
CVE-2026-4800	lodash 4.17.20	HIGH	Code Injection via _.template
CVE-2025-13465	lodash 4.17.20	MEDIUM	Prototype Pollution
CVE-2022-23539	jsonwebtoken 8.5.1	HIGH	Insecure key types
CVE-2022-23540	jsonwebtoken 8.5.1	MEDIUM	Signature bypass (none algo)
CVE-2022-23541	jsonwebtoken 8.5.1	MEDIUM	RSA to HMAC forgery
CVE-2024-29041	express 4.17.1	MEDIUM	Open Redirect

Le scan de l'image `nginx:1.21` révèle **47 vulnérabilités** : 3 CRITICAL, 12 HIGH, 18 MEDIUM, 14 LOW.

Espace réservé pour la Figure 4.6

Capture d'écran : Résultats du scan Trivy image sur nginx:1.21

Figure 4.6: Résultats du scan Trivy montrant les CVEs de l'image nginx:1.21

4.3.4 DAST avec OWASP ZAP

OWASP ZAP effectue un scan en boîte noire de l'application déployée :

Tableau 4.11: Findings OWASP ZAP sur l'application

Finding	Règle	Sévérité
Stored XSS (dangerouslySetInnerHTML)	40012	HIGH
Missing Content-Security-Policy	10038	MEDIUM
Missing X-Frame-Options (Clickjacking)	10020	MEDIUM
Directory Browsing Enabled	10033	MEDIUM
Server Version Leaked (nginx/1.21.6)	10036	LOW
Missing HSTS	10035	LOW
Missing X-Content-Type-Options	10021	LOW

Après durcissement de `nginx.conf` (ajout des headers de sécurité, désactivation de `server_tokens` et `autoindex`) : **0 FAIL, 0 WARN**.

Espace réservé pour la Figure 4.7

Capture d'écran : Rapport OWASP ZAP (avant durcissement)

Figure 4.7: Rapport OWASP ZAP montrant les findings avant durcissement nginx

4.3.5 GitOps avec Argo CD

4.3.5.1 Configuration de l'Application Argo CD

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: login-page-replicator
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/nnaffati/login-page-replicator
    targetRevision: HEAD
    path: k8s/app
  destination:
    server: https://kubernetes.default.svc
    namespace: app
  syncPolicy:
    automated:
      prune: true      # Supprime les ressources obsoletes
      selfHeal: true   # Corrige tout drift manuel
    syncOptions:
      - CreateNamespace=true
```

Listing 4.5: Configuration Application Argo CD

Espace réservé pour la Figure 4.8

Capture d'écran : Dashboard Argo CD montrant l'application synchronisée

Figure 4.8: Dashboard Argo CD montrant l'état synchronisé de l'application

4.3.6 Réseau Zero Trust — NetworkPolicies

Sans NetworkPolicies, tous les pods communiquent librement. La politique appliquée : **tout est interdit par défaut, chaque flux autorisé est déclaré explicitement**.

Tableau 4.12: Matrice des flux réseau autorisés

Source	Destination	Port	Politique
`testing` pods	`app` pods	TCP 80	allow-testing-to-app
PostSync hook	`app` pods	TCP 80	allow-playwright-hook
`observability`	`app` sidecar	TCP 9113	allow-prometheus-scrape
`security` DaemonSet	Wazuh VM	TCP 1514/1515	allow-wazuh-egress
`app` pods	kube-dns	UDP/TCP 53	allow-dns-egress
kubelet	`app` pods	TCP (probes)	allow-kubelet-probes

4.3.7 Rétrospective Sprint 3

Tableau 4.13: Rétrospective Sprint 3

Catégorie	Détail
Ce qui a bien marché	Tous les scans de sécurité détectent les vulnérabilités attendues, Argo CD auto-sync opérationnel
Difficultés	Gestion des faux positifs Trivy/ZAP, orchestration des NetworkPolicies avec les PostSync hooks
Livrable	Sécurité SAST/SCA/DAST intégrée, GitOps opérationnel, réseau Zero Trust

4.4 Sprint 4 : Tests E2E, Observabilité, SIEM et Bilan

4.4.1 Objectif et Backlog du Sprint

Tableau 4.14: Backlog du Sprint 4

User Story	Tâche	Statut
US-10 : Playwright E2E	Écrire 6 scénarios Playwright (@t1 à @t6)	Terminé
US-10	Configurer l'exécution in-cluster (PostSync + standalone Job)	Terminé
US-11 : Observabilité	Déployer Prometheus + Grafana dans le namespace observability	Terminé
US-11	Configurer le ServiceMonitor pour le sidecar nginx-exporter	Terminé
US-12 : Wazuh SIEM	Déployer le DaemonSet Wazuh Agent dans le namespace security	Terminé
US-12	Connecter les agents au manager Wazuh sur la VM	Terminé

4.4.2 Tests Unitaires avec Vitest

Les tests unitaires couvrent les composants React critiques :

- **TodoItem** : rendu du texte, toggle completed, suppression ;
- **AuthContext** : login/logout, persistence de session ;
- **TodoDashboard** : CRUD complet, filtrage, compteurs.

La couverture de code est supérieure à 80% grâce au Quality Gate SonarCloud.

Espace réservé pour la Figure 4.9

Capture d'écran : Rapport de couverture Vitest

Figure 4.9: Rapport de couverture des tests unitaires Vitest

4.4.3 Tests E2E Playwright — 6 Scénarios

Tableau 4.15: Les 6 scénarios de tests Playwright E2E

Tag	Scénario	Ce que ça valide
@t1	Credentials invalides → message d'erreur	Authentification, gestion erreurs
@t2	Login → dashboard vide affiché	Flux d'authentification complet
@t3	Login → Logout → retour formulaire	Cycle auth complet
@t4	Session persiste après rechargement	Persistance localStorage
@t5	Todos persistent après rechargement	Persistance des données
@t6	Ajouter, compléter, supprimer une tâche	CRUD complet

Les tests sont exécutés en trois modes :

- **In-cluster (PostSync)** : jobs Kubernetes créés par Argo CD après chaque déploiement ;
- **Local CI (matrice)** : GitHub Actions exécute Playwright avec `-grep @t1..@t6` ;
- **Standalone Job** : `k8s/testing/playwright-job.yaml` pour tests manuels.

Espace réservé pour la Figure 4.10

Capture d'écran : Résultats des 6 tests Playwright E2E

Figure 4.10: Résultats des tests Playwright E2E (6/6 passed)

4.4.4 Observabilité avec Prometheus et Grafana

Le flux de collecte des métriques :

1. Le sidecar `nginx-exporter` (port 9113) expose les métriques Nginx ;
2. Le `ServiceMonitor` scrape ces métriques toutes les 30 secondes ;
3. Prometheus stocke les séries temporelles ;
4. Grafana interroge Prometheus pour les tableaux de bord.

Tableau 4.16: Métriques Nginx collectées et leur utilité sécurité

Métrique	Description	Utilité Sécurité
nginx_connections_active	Connexions actives	Détection de DDoS
nginx_http_requests_total	Requêtes par status code	Spike de 4xx = attaque de scan
nginx_connections_accepted	Connexions acceptées total	Baseline trafic normal
nginx_connections_handled	Connexions traitées	Performance et disponibilité

Espace réservé pour la Figure 4.11

Capture d'écran : Dashboard Grafana avec métriques Nginx

Figure 4.11: Dashboard Grafana montrant les métriques Nginx en temps réel

4.4.5 Sécurité Runtime — Wazuh SIEM

Wazuh est un SIEM open-source qui fournit : détection d'intrusion (HIDS), monitoring de l'intégrité des fichiers (FIM), conformité réglementaire, et corrélation d'événements.

L'agent Wazuh est déployé comme **DaemonSet** dans le namespace `security`, collectant les logs depuis `/var/log/containers/` et les envoyant via TCP 1514 vers le manager sur la VM GCE.

> **Limite GKE Autopilot :** Autopilot bloque `hostPID`, `hostNetwork` et les conteneurs privilégiés, limitant le monitoring kernel de Wazuh. Les logs conteneurs et l'inventaire restent fonctionnels.

Espace réservé pour la Figure 4.12

Capture d'écran : Dashboard Wazuh montrant les agents connectés

Figure 4.12: Dashboard Wazuh avec 4 agents opérationnels

4.4.6 Gouvernance IAM et Identité

Tableau 4.17: Identités et rôles IAM du projet

Identité	Type	Rôles GCP	Usage
gke-app-sa	GSA	artifactregistry.reader	Pull d'images depuis les nœuds
github-actions-sa	GSA	container.developer, artifactregistry.writer	Pipeline CI/CD
playwright-runner	KSA	Via Workload Identity	Jobs Playwright en cluster

4.4.7 Rétrospective Sprint 4

Tableau 4.18: Rétrospective Sprint 4

Catégorie	Détail
Ce qui a bien marché	6/6 tests Playwright passent, Grafana dashboards opérationnels, Wazuh agents connectés
Difficultés	Limitations GKE Autopilot pour Wazuh kernel monitoring, RBAC pour les PostSync hooks
Livrable	Plateforme DevSecOps complète et opérationnelle

4.5 Bilan Global : Résultats et Comparaison Avant/Après

4.5.1 Tableau de Bord de Sécurité

Tableau 4.19: Bilan sécurité avant et après la transformation DevSecOps

Domaine	Avant (Jenkins)	Après (DevSecOps)
SAST	Absent	SonarCloud détecte XSS, hotspots
SCA	Absent	Trivy FS : 9 CVEs sur 3 packages
Container Scan	Absent	Trivy Image : 47 CVEs nginx:1.21
IaC Scan	Absent	Trivy Config : 8 misconfigs K8s
DAST	Absent	OWASP ZAP : 10 findings sécurité
Secrets Scan	Absent	Trivy : secrets ENV + .env détectés
Runtime Security	Absent	Wazuh : 4 agents actifs
Réseau	Tout ouvert	Zero Trust : 12 NetworkPolicies
Auth CI	Clés JSON statiques	WIF OIDC : zéro clé persistante
Tests	Unit tests Jenkins	Unit + E2E + 6 scénarios Playwright
Infrastructure	ClickOps	Terraform IaC : 100% reproductible
Déploiement	Scripts manuels	Argo CD GitOps : auto-sync + self-heal

4.5.2 KPIs Techniques Mesurés

Tableau 4.20: Indicateurs clés de performance avant/après

KPI	Avant	Après
Vulnérabilités détectées avant prod	0	9+ CVEs + 10 findings ZAP
Temps de détection d'une CVE critique	Jamais	< 5 min (sur push)
Infrastructure reproductible	Non	Oui (terraform apply)
Rollback automatique	Non	Oui (git revert + Argo CD)
Tests E2E automatisés	0	6 scénarios in-cluster
Couverture tests unitaires	40%	> 80% (gate SonarCloud)
Auth CI sans clé statique	Non	Oui (WIF OIDC)
Namespaces avec isolation réseau	0	5 (100% du cluster)
SIEM actif	Non	Oui (Wazuh 4 agents)

4.5.3 Alignement aux Frameworks de Sécurité

Tableau 4.21: Alignement aux frameworks de sécurité

Framework	Couverture Actuelle	Objectif
OWASP Top 10	A03, A05, A06, A09 couverts	A01, A02, A08 à ajouter
NIST SSDF	PW.1 (Build), RV.1 (Scan)	PW.4 (Signing), PO.3 (Reqs)
CIS Controls v8	Config, Vuln Mgmt, AppSec	Inventory, Logging
OWASP SAMM	Secure Build, Sec Testing	Secure Architecture, Operations

Conclusion

Ce chapitre a présenté l'implémentation complète du projet à travers quatre sprints Agile : le Sprint 1 a posé les fondations avec Terraform et GKE, le Sprint 2 a construit la containerisation et le pipeline CI/CD avec authentification keyless, le Sprint 3 a intégré les couches de sécurité (SAST, SCA, DAST) et le déploiement GitOps avec réseau Zero Trust, et le Sprint 4 a complété la plateforme avec les tests E2E, l'observabilité et le SIEM. Le bilan quantitatif démontre la suppression de toutes les lacunes sécuritaires identifiées au chapitre 1 et confirme que la problématique posée en début de stage a été pleinement résolue.

Conclusion et Perspectives

Ce projet de fin d'études avait pour ambition de répondre à une problématique concrète : *comment intégrer la sécurité à chaque étape d'une chaîne de livraison logicielle cloud-native, sans sacrifier la vélocité ?* La conclusion générale revient sur l'ensemble de la démarche, évalue l'atteinte des objectifs, analyse les difficultés rencontrées, mesure les apports de part et d'autre, et ouvre sur des perspectives d'approfondissement.

Récapitulation de la Démarche

La démarche suivie tout au long de ce projet s'est organisée en quatre étapes progressives et complémentaires. Un premier travail d'analyse a permis de cartographier précisément les lacunes du pipeline Jenkins existant : absence totale de scans de sécurité, clés d'authentification statiques, infrastructure non reproductible et absence de monitoring. Sur la base de cette analyse, les besoins fonctionnels et non fonctionnels ont été formalisés, puis une vision cible DevSecOps a été conçue autour de huit principes directeurs et d'une architecture en six couches.

L'implémentation, organisée en quatre sprints Agile, a couvert : le provisionnement de l'infrastructure GCP via Terraform (Sprint 1), la containerisation et le pipeline CI/CD avec authentification keyless (Sprint 2), l'intégration des couches de sécurité SAST/SCA/DAST et le déploiement GitOps avec réseau Zero Trust (Sprint 3), et enfin les tests E2E, l'observabilité et la surveillance runtime (Sprint 4).

Présentation des Résultats

Les résultats obtenus répondent directement à chaque lacune identifiée dans la problématique initiale. Le tableau 4.22 synthétise les indicateurs clés.

Difficultés Rencontrées

Contraintes GKE Autopilot : Le mode Autopilot bloque les capacités privilégiées des pods (`hostPID`, `hostNetwork`). Ces restrictions ont limité la surveillance kernel de Wazuh. La solution retenue est de déployer le manager Wazuh sur une VM Compute Engine dédiée et de connecter les agents via TCP 1514.

Orchestration des tests E2E en GitOps : L'intégration de Playwright comme

Tableau 4.22: Bilan quantitatif de la transformation DevSecOps

Indicateur	Avant	Après
CVEs détectées (npm)	0 (non scanné)	9 CVEs dont 4 HIGH
CVEs détectées (image)	0 (non scanné)	47 CVEs dont 3 CRITICAL
Findings OWASP ZAP	0 (non testé)	10 findings dont 1 HIGH (XSS)
Secrets détectés	0 (non scanné)	3 secrets exposés identifiés
Auth CI/CD	Clé JSON statique	OIDC éphémère (WIF)
Infrastructure	ClickOps manuel	Terraform IaC 100%
Tests E2E	Absents	6 scénarios Playwright in-cluster
Coverage tests unitaires	Non mesurée	70 %+ (Vitest)
NetworkPolicies	Aucune	12 règles default-deny
SIEM actif	Absent	Wazuh 4 agents opérationnels
ZAP FAIL count	Non applicable	0 (après durcissement nginx)

PostSync hooks Argo CD a nécessité une conception minutieuse des RBAC et des NetworkPolicies pour permettre aux pods de tests d'accéder à l'application dans le respect du principe Zero Trust.

Gestion des faux positifs : Les scanners génèrent des faux positifs qui nécessitent un tri manuel. La mise en place de fichiers d'exception (`.trivyignore`, règles SonarCloud) a permis de filtrer le bruit sans masquer les vulnérabilités réelles.

Workload Identity Federation : La configuration initiale du mécanisme OIDC entre GitHub Actions et GCP a exigé une maîtrise des concepts d'identité fédérée, de pools de fournisseurs d'identité et de conditions d'attribut — concepts peu documentés pour ce cas d'usage précis.

Apports du Stage

Apports à l'entreprise : Le projet a livré à Sopra Steria une plateforme DevSecOps opérationnelle et documentée, un référentiel de bonnes pratiques, une démonstration concrète de l'intégration SAST/SCA/DAST dans GitHub Actions, et des templates Terraform/Kubernetes réutilisables pour d'autres projets.

Apports pour l'élève-ingénieur : Ce stage a permis d'acquérir des compétences transversales rares combinant le cloud engineering, la sécurité applicative, l'orchestration Kubernetes et l'ingénierie des systèmes distribués.

Perspectives d'Approfondissement

Court terme :

Tableau 4.23: Compétences acquises durant le stage

Domaine	Compétences Acquises
Cloud Engineering	GCP, GKE Autopilot, Terraform, Artifact Registry, IAM
DevSecOps	GitHub Actions, Argo CD, GitOps, Shift-Left Security
Sécurité applicative	Trivy, OWASP ZAP, SonarCloud, gestion des CVEs
Kubernetes	Deployments, NetworkPolicies, RBAC, HPA, Jobs, Ingress
Observabilité	Prometheus, Grafana, ServiceMonitor, métriques nginx
Développement	React 18, TypeScript, Vite, Vitest, Playwright E2E
Méthode	Analyse de risque, architecture en couches, documentation

- Intégration de **Gitleaks** en hook pre-commit pour détecter les secrets avant qu'ils n'atteignent le dépôt Git ;
- Signature des images Docker avec **cosign** et vérification à l'admission via **Kyverno** ;
- Ajout de règles Kyverno pour enforcer les contraintes de sécurité pod-level.

Moyen terme :

- Migration vers **Istio Service Mesh** pour une mTLS automatique entre services et une observabilité réseau L7 ;
- Génération et publication d'un **SBOM** (*Software Bill of Materials*) via Syft à chaque release ;
- Intégration de **Falco** pour la détection d'anomalies comportementales syscall dans les pods.

Long terme :

- Atteindre le niveau **SLSA 3** avec des preuves de provenance vérifiables ;
- Déploiement multi-région avec stratégie de *disaster recovery* automatisée ;
- Mise en conformité formelle **ISO 27001** en s'appuyant sur les artefacts d'audit générés automatiquement par la plateforme.

Message Final

Ce projet illustre une vérité fondamentale du développement logiciel moderne :

« La sécurité n'est pas un obstacle à la vélocité — c'est un enabler de confiance. »

Une équipe qui automatise ses contrôles de sécurité livre plus vite, avec plus de confiance, et consacre moins de temps à gérer des incidents. Le DevSecOps n'est pas une contrainte imposée par les équipes sécurité — c'est une pratique d'ingénierie mature qui bénéficie à l'ensemble de l'organisation.

Appendix A

Commandes de Déploiement

1.1 Provisionnement de l'Infrastructure

```
cd terraform/
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

Listing A.1: Provisionnement Terraform

1.2 Deploiement des Ressources Kubernetes

```
kubectl apply -f k8s/namespaces.yaml
kubectl apply -f k8s/app/
kubectl apply -f k8s/security/
kubectl apply -f k8s/argocd/app-application.yaml
```

Listing A.2: Application des manifests Kubernetes

1.3 Verification de l'Etat Argo CD

```
kubectl get applications -n argocd
kubectl describe application login-page-replicator -n argocd
```

Listing A.3: Verification de la synchronisation Argo CD

1.4 Lancement des Tests E2E

```
SHORT_SHA="<sha>"
sed "s|IMAGE_TAG|${SHORT_SHA}|g" k8s/testing/playwright-job.yaml \
  | kubectl apply -f -
kubectl logs -n testing -l app=playwright --tail=100 -f
```

Listing A.4: Lancement manuel des tests Playwright

1.5 Consultation des Logs Wazuh

```
kubectl logs -n security -l app=wazuh-agent --tail=50
```

Listing A.5: Consultation des logs de l'agent Wazuh

Appendix B

Variables GitHub Actions et Ports des Services

Sommaire

1 Provisionnement de l'Infrastructure 71

2 Deploiement des Ressources Kubernetes 71

3 Verification de l'Etat Argo CD 71

4 Lancement des Tests E2E 71

5 Consultation des Logs Wazuh 71

2.1 Secrets et Variables GitHub Actions

Tableau B.1: Variables et secrets requis dans GitHub Actions

Secret/Variable	Description	Source
WIF_PROVIDER	ID du pool Workload Identity Federation	Terraform output
WIF_SERVICE_ACCOUNT	Email du GSA GitHub Actions	Terraform output
SONAR_TOKEN	Token d'authentification SonarCloud	SonarCloud UI
DAST_TARGET_URL	URL cible pour le scan OWASP ZAP	Cloud Run URL
GKE_CLUSTER	Nom du cluster GKE	devops-cluster
GKE_ZONE	Region du cluster GKE	europe-west1

2.2 Ports et Services de la Plateforme

Tableau B.2: Ports et services de la plateforme DevSecOps

Service	Port	Protocole	Namespace	
App nginx	80	HTTP	app	
nginx-exporter (Prometheus)	9113	HTTP	app	
Wazuh Manager (events)	1514	TCP	VM GCE	
Wazuh Manager (enrollment)	1515	TCP	VM GCE	
Wazuh Dashboard	443	HTTPS	VM GCE	
Prometheus	9090	HTTP	observability	
Grafana	3000	HTTP	observability	

Appendix C

Glossaire des Termes Techniques

Sommaire

1 Secrets et Variables GitHub Actions 73

2 Ports et Services de la Plateforme 73

CI/CD *Continuous Integration / Continuous Delivery* — automatisation du build, des tests et du deploiement logiciel.

CVE *Common Vulnerabilities and Exposures* — identifiant unique d'une vulnerabilite de securite publiee.

DAST *Dynamic Application Security Testing* — test de securite sur une application en cours d'execution (boite noire).

DaemonSet Ressource Kubernetes garantissant qu'un pod tourne sur chaque noeud du cluster.

GitOps Paradigme operationnel ou Git est la source unique de verite pour l'etat de l'infrastructure et des applications.

GSA *Google Service Account* — identite machine dans Google Cloud Platform.

HPA *Horizontal Pod Autoscaler* — scaling automatique horizontal des pods Kubernetes en fonction de metriques de charge.

IaC *Infrastructure as Code* — description et gestion de l'infrastructure dans du code versionne (Terraform, CloudFormation...).

KSA *Kubernetes Service Account* — identite machine dans Kubernetes.

OIDC *OpenID Connect* — protocole d'authentification base sur OAuth2 utilisant des tokens JWT.

SAST *Static Application Security Testing* — analyse statique du code source sans execution.

SCA *Software Composition Analysis* — audit des dependances et composants tiers pour detecter des CVEs.

SIEM *Security Information and Event Management* — centralisation et correlation d'evenements de securite en temps reel.

SLSA *Supply-chain Levels for Software Artifacts* — framework d'integrite des artefacts logiciels.

WIF *Workload Identity Federation* — authentification keyless entre GitHub Actions et GCP via OIDC.

XSS *Cross-Site Scripting* — injection de code JavaScript malveillant dans une page web.

Zero Trust Modele de securite reseau : ne jamais faire confiance implicitement, toujours verifier. Tout flux reseau est interdit par defaut.

Résumé

Ce rapport présente la conception et l'implémentation d'une plateforme **DevSecOps** complète autour d'une application web moderne développée avec React, TypeScript et Vite, déployée sur **Google Kubernetes Engine (GKE)** en mode Autopilot. Le projet part d'une situation initiale où l'équipe utilisait uniquement **Jenkins** comme pipeline d'intégration continue pour exécuter des tests unitaires, sans aucune considération de sécurité intégrée.

L'objectif principal est de transformer cette approche naïve en une **chaîne de valeur logicielle sécurisée de bout en bout**, en intégrant : l'Infrastructure as Code avec Terraform, le GitOps avec Argo CD, l'analyse de vulnérabilités avec Trivy et OWASP ZAP, la sécurité runtime avec Wazuh SIEM, l'observabilité avec Prometheus et Grafana, et un modèle d'identité sans clés statiques via Workload Identity Federation.

Le résultat est une plateforme reproductible, auditable et directement applicable à des contextes de production réels.

Mots clés : DevSecOps, Kubernetes, GKE, Terraform, Argo CD, Trivy, OWASP ZAP, Wazuh, CI/CD, GitOps, Zero Trust, Workload Identity, SAST, DAST, SCA, GitHub Actions, Prometheus, Grafana.

Abstract

This report presents the design and implementation of a complete **DevSecOps** platform built around a modern web application developed with React, TypeScript and Vite, deployed on **Google Kubernetes Engine (GKE)** in Autopilot mode. The project starts from an initial situation where the team was using only **Jenkins** as a continuous integration pipeline to run unit tests, with no integrated security considerations whatsoever.

The main objective is to transform this naive approach into a **secure end-to-end software value chain**, integrating: Infrastructure as Code with Terraform,